Hackyard Build 2026 Submission · Team TwinForge · IIT Guwahati

OrbitScore: A Source-Linked Pre-Review Copilot for Spacecraft Debris-Mitigation Evidence

Team: TwinForge · IIT Guwahati

Submission context: Hackyard Build 2026

Document status: Submission-ready research paper; backend and frontend verification completed on the packaged source tree.

> ***Safety boundary.*** *OrbitScore is a decision-support and pre-review workflow. It is not legal certification, a regulator, a collision-prediction system, an operator-facing conjunction-alert service, a maneuver recommender, or a replacement for NASA Debris Assessment Software. Its profiles are inspired interpretations; they do not certify compliance. Synthetic fixtures and counterfactuals are illustrative, not mission evidence.*

Abstract

Small-spacecraft teams assemble orbit, propulsion, post-mission disposal, passivation, and source-document evidence before an expert or regulatory review. These inputs are commonly distributed across PDFs, spreadsheets, and design notes, making missing proof and conflicting values difficult to spot. OrbitScore is a prototype source-linked pre-review copilot that stores candidate fields with locators, requires human confirmation, evaluates confirmed facts against one explicitly selected policy profile, records deterministic findings, and exports a cited review packet only after a human review gate.

The implemented application separates assistive evidence organization from deterministic decision logic. apps/api/app/engine.py evaluates the selected profile; provenance.py computes and verifies a SHA-256 event chain; export.py generates a cited HTML/JSON packet; frontend-service/api.ts dynamically evaluates rules and persists offline/mock mutations without fixture-status shortcuts. Four versioned profiles are included. The OrbitScore/MSRI band is an evidence-prioritization index, not a safety probability, and excludes collision-risk terms. Verification on this package: 7 Python tests passed, 5 Vitest tests passed, TypeScript typecheck and production build passed, and the generated matrix records 12 fixture/profile evaluations with a valid provenance chain.

Keywords: orbital debris; evidence readiness; post-mission disposal; provenance; human-in-the-loop; counterfactual explanation; deterministic rules

1. Motivation and design objective

Orbital debris mitigation is a systems-engineering and sustainability concern [1–3]. The practical problem addressed here is narrower than orbital-environment modeling: a student or smallsat team may possess an orbit concept, disposal plan, propulsion data, and several documents but lack a trace connecting each claim to its source, date, scope, unit, and reviewer action. A disposal lifetime copied into a spreadsheet may conflict with a datasheet; an extracted value may be unconfirmed; or a rule may be applied outside its jurisdictional scope.

OrbitScore is designed for the handoff immediately before specialist analysis, filing, or an authorized assessment tool:

```
mission facts → evidence + locators → selected profile → deterministic findings
→ gaps and reviewer actions → bounded counterfactual → cited pre-review packet
```

The product thesis is traceability rather than prediction. It should answer: **What is known? What is only asserted? What evidence is missing? Which scoped profile was selected? What declared change would alter the review finding?**

2. Scope-controlled policy profiles

Profiles are stored as versioned rule fixtures. A rule carries a stable ID, version, profile ID, jurisdiction scope, source citation, required fields, scope condition, evaluation logic, evidence requirement, critical-gap flag, remediation lever, and limitation notice. OrbitScore never silently merges rules from different profiles.

Profile	Configured purpose	Boundary
`india-dfsm-inspired`	Checklist inspired by India's Debris-Free Space Missions initiative, including the stated greater-than-99% post-mission-disposal probability, less-than-five-year remaining orbital life in the stated context, trackability, identifiability, maneuverability, passivation, and the 400 ± 30 km human-spaceflight-band avoidance language where applicable	An inspired, source-linked checklist; not ISRO endorsement or a universal Indian certificate
`nasa-std-8719-14c-inspired`	NASA-STD-8719.14C-inspired checks, including scoped Requirement 4.3-1 wording for planned debris of at least 1 mm passing through LEO, the stated 25-year lifetime condition and object-years condition, plus separately modeled passivation, disposal, collision-probability, and uncontrolled-reentry checks where configured	A NASA-scoped interpretation fixture; not NASA compliance, NASA approval, or a replacement for NASA DAS
`fcc-22-74-inspired`	FCC 22-74-inspired check for covered LEO space stations below 2,000 km using uncontrolled atmospheric reentry, including a five-year post-mission disposal lifetime and trackability/maneuverability disclosure checks	Not a universal five-year law and not an FCC licensing decision
`internal-pre-review`	Baseline evidence checklist for student/smallsat design reviews	Internal engineering triage; not a legal or regulatory determination

The FCC language is intentionally narrow: the five-year condition is only presented for the covered LEO/uncontrolled-reentry scope represented by the configured profile. The NASA 4.3-1 wording is retained as a scoped planned-debris requirement, not as a universal rule for every object or mission. OrbitScore does not infer a global population from these profiles.

3. System architecture and evidence states

The architecture separates document assistance, deterministic evaluation, provenance, and human review.

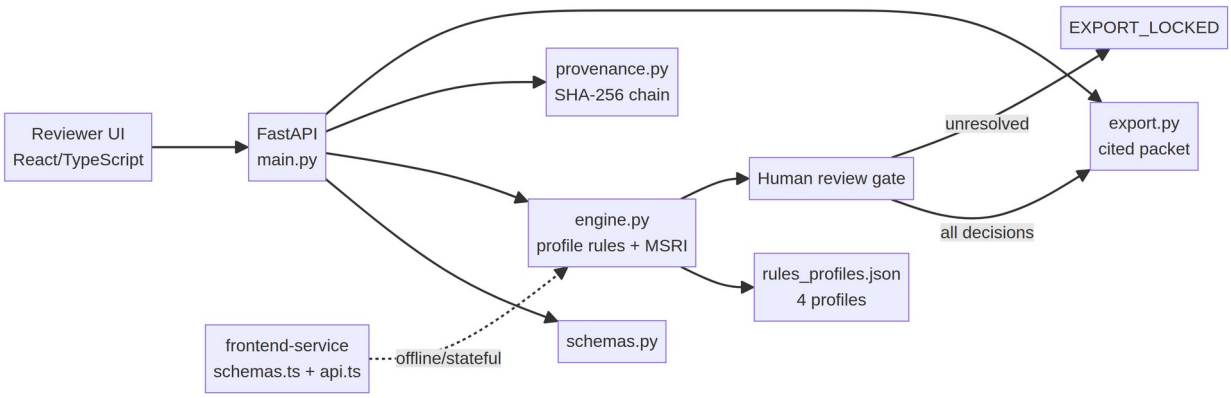


Figure 1. Implemented decision boundary. Evidence fields retain source locators and confirmation state; the deterministic evaluator consumes the selected profile and confirmed values. The package test suites and production build passed as summarized in Section 7.

3.1 Evidence lifecycle

An artifact stores filename, MIME type, content hash, source title, source date, retrieval date, and page/span locator where available. Extraction creates a candidate `ExtractedField` with value, unit, method, confidence, and `human_confirmed=false`. A reviewer can correct or confirm the field; the correction is a new auditable mutation, not a silent replacement. Conflicting source values remain visible and are not averaged.

The evaluator returns one of five explicit states:

- **`PASS_UNDER_PROFILE`** — confirmed inputs and required evidence satisfy the selected rule.
- **`WARNING`** — the rule evaluates but a stated limitation, age, or scope caveat remains.
- **`NEEDS_EVIDENCE`** — a required claim is absent, unconfirmed, stale, or lacks provenance.
- **`CONFLICT`** — two accepted sources disagree; a reviewer must resolve the conflict.
- **`NOT_EVALUATED`** — profile scope or available inputs do not support a defensible evaluation.

A critical gap caps the readiness band. The band is an evidence-prioritization aid only; it cannot override `NEEDS_EVIDENCE`, `CONFLICT`, or `NOT_EVALUATED`.

3.2 OrbitScore/MSRI band

The band combines transparent evidence-completeness and unresolved-critical-finding indicators so a reviewer can prioritize work. It is not a calibrated probability, casualty expectation, conjunction likelihood, or legal status. No `S_coll` term is included in the score. A score or band never converts missing information into zero risk and never outputs “approved.”

4. Deterministic evaluation and counterfactuals

The implemented backend is a FastAPI service. main.py exposes the documented routes; schemas.py validates request payloads; engine.py loads a selected profile and applies deterministic operators to confirmed fields; provenance.py appends and verifies an event hash chain; and export.py builds a cited HTML/JSON packet. The frontend contract is frontend-service/schemas.ts (ten domain entities and runtime validation) plus frontend-service/api.ts (dynamic profile evaluation, persistent mutations, in-memory test fallback, and reset support).

A controlled counterfactual changes only an explicitly declared lever, such as a disposal-lifetime assumption or presence of disposal evidence. It reports the before value, proposed value, affected rule, before/after state, and assumptions. For example, an 8.4-year synthetic disposal-lifetime value may remain above a five-year configured threshold but below a 25-year configured threshold; the resulting difference is a property of the selected rule fixtures, not proof of regulatory acceptance. A proposed value of 4.2 years may illustrate a threshold flip in a synthetic comparison, but it is not an atmospheric-decay prediction, a burn command, or a flight recommendation.

A physically meaningful disposal analysis requires mission-specific dynamics, mass properties, area-to-mass ratio, atmospheric assumptions, propulsion performance, execution reliability, and evidence. OrbitScore does not claim to provide that analysis.

5. Provenance and human review

The demo chain records each event type and payload, sequence, previous hash, and computed SHA-256 hash. It detects later alteration under the controlled storage model; it does not make the database immutable or legally authoritative and is not a signed production audit log. verifyProvenanceChain() returns the first broken sequence number so a reviewer can locate tampering precisely.

The export gate is intentionally conservative. `exportPacket()` must return `EXPORT_LOCKED` until every finding has a persisted `ReviewDecision` of `APPROVED`, `REJECTED`, or `WAIVED_WITH_NOTE`. Human review remains mandatory for profile selection, conflicts, critical gaps, assumptions, and any action that could alter mission design.

The automation boundary is **inspired by** the human-automation discussion of Parasuraman, Sheridan, and Wickens [16]; that citation is inspiration, **not certification against their taxonomy**. OrbitScore may assist with parsing, normalization, trace display, and packet assembly. A qualified human decides whether evidence is adequate and what to do next.

6. Synthetic case studies

The following cases are deliberately synthetic and illustrate workflow behavior only.

6.1 Complete evidence case

mission_complete is evaluated under nasa-std-8719-14c-inspired with PASS_UNDER_PROFILE for all seven configured checks (including NASA-4.6-LIFETIME); its MSRI band is GREEN. Under internal-pre-review, six checks pass and INT-EVIDENCE is NEEDS_EVIDENCE, so the band is AMBER. These are outputs on synthetic inputs, not compliance findings.

6.2 Gap-heavy case

mission_gap_heavy leaves disposal lifetime, passivation, and other policy fields unconfirmed. Under nasa-std-8719-14c-inspired all seven configured findings are NEEDS_EVIDENCE; under fcc-22-74-inspired the lifetime finding FCC-22-74-5YR is NEEDS_EVIDENCE. mission_conflict contains two confirmed disposal-lifetime values; NASA-4.6-LIFETIME is CONFLICT. The engine does not average the conflicting values.

6.3 Profile-switch case

The verified 8.4-year mission_complete lifetime yields PASS_UNDER_PROFILE for NASA-4.6-LIFETIME under nasa-std-8719-14c-inspired (25-year threshold), but WARNING for FCC-22-74-5YR under fcc-22-74-inspired (five-year condition; the synthetic mission is in configured scope). Under india-dfsm-inspired, IN-DFSM-ORBIT-LIFE is WARNING because 8.4 years is not below five years. Each result is tied to its selected profile and version; none establishes an authority decision.

6.4 Bounded counterfactual case

A reviewer proposes a synthetic evidence-backed lifetime value or an evidence-presence change. The engine evaluates the mission using the proposed value and reports the targeted finding before and after. The result is labeled illustrative synthetic counterfactual and cannot be read as a causal guarantee, safety claim, or maneuver recommendation.

7. Implementation status and verification results

Capability	Implemented location	Submission status in this workspace	Boundary
FastAPI routes and health endpoint	`apps/api/app/main.py`	Implemented; exercised by backend tests	API orchestration is not regulator integration
Pydantic API request models	`apps/api/app/schemas.py`	Implemented; 7 pytest tests pass	Invalid/unscoped inputs should fail closed
Four-profile deterministic evaluator and MSRI band	`apps/api/app/engine.py`, `rules_profiles.json`	Implemented; 12 matrix evaluations generated	No `S_coll`; no physical risk probability
SHA-256 provenance chain and verifier	`apps/api/app/provenance.py`	Implemented; tamper test passes	Tamper-evident, not legally immutable
Cited HTML/JSON packet and review gate	`apps/api/app/export.py`	Implemented; export-gate test passes	All findings need persisted decisions
Typed entities and stateful offline adapter	`frontend-service/schemas.ts`, `frontend-service/api.ts`	Implemented; 5 Vitest tests pass	Mock mode must evaluate dynamically, not replay statuses
Backend tests	`apps/api/tests/test_evaluator.py`	7 passed	Must cover three fixtures, four profiles, counterfactuals, gate, tamper detection
Frontend tests	`tests/api.test.ts`	5 passed	Must cover profile switching, persistence, counterfactuals

The exact verification commands are:

```
pytest -q apps/api/tests/test_evaluator.py
npm test -- --run
```

Verification completed on 2026-10-20 from the packaged source tree: `pytest -q` reported 7 passed; `npm test` reported 5 passed; `npm run typecheck` and `npm run build` completed successfully. `python -m app.verification` produced 12 deterministic fixture/profile evaluations and verified all 12 events in the SHA-256 chain. The detailed output matrix is included in fixtures/evaluation_verification_report.json.

8. Validation plan

The minimum reproducibility matrix is:

1. Evaluate `mission_complete`, `mission_gap_heavy`, and `mission_conflict` under all four profiles.
2. Confirm that changing the selected profile changes only profile-dependent findings and preserves provenance.
3. Confirm that unconfirmed values cannot produce a pass for rules requiring confirmed evidence.
4. Confirm that conflicts are visible and never silently averaged.
5. Confirm that a bounded counterfactual changes only declared inputs and affected findings.
6. Confirm that all state mutations persist in local storage with an in-memory fallback for Node/Vitest tests.
7. Confirm that export remains `EXPORT_LOCKED` until every finding has a review decision.
8. Tamper with one stored provenance event and report the exact first broken sequence.
9. Inspect the exported packet for profile ID/version, citations, assumptions, reviewer decisions, synthetic labels, and limitations.

Scientific validation, regulatory acceptance, operational conjunction performance, generalization across jurisdictions, and measured organizational time savings remain future work.

9. Limitations and ethical boundaries

- OrbitScore is a pre-review assistant, not a compliance authority or legal certification tool.
- Profile names ending in `-inspired` identify configured interpretations, not agency endorsement.
- FCC 22-74 is represented only for covered LEO space stations below 2,000 km using uncontrolled atmospheric reentry.
- NASA Requirement 4.3-1 is described as scoped planned-debris wording; OrbitScore does not turn it into a universal rule.
- Synthetic data is not mission evidence; counterfactuals are illustrative.
- The system does not predict collisions, casualty risk, orbital decay, or conjunctions.
- Public orbital data, if later connected, would require an epoch, age, source, and usage label.
- Hashes support traceability but do not create legal immutability.
- A readiness band is not a safety probability and cannot override a critical evidence gap.

10. Conclusion

OrbitScore's contribution is a defensible handoff layer: source-linked facts, explicit evidence states, profile-scoped deterministic checks, bounded what-if comparisons, tamper-evident provenance, and a human-gated review packet. Its credibility depends on maintaining the boundary between assistive extraction and decision logic, labeling interpretations and synthetic examples honestly, and publishing test output only after the promised backend and frontend suites have actually run. The result is intentionally modest: a reproducible pre-review workflow that helps experts see what is known, what is missing, and what must be resolved next.

References

[1] D. J. Kessler and B. G. Cour-Palais, “Collision Frequency of Artificial Satellites: The Creation of a Debris Belt,” *Journal of Geophysical Research*, 1978. https://doi.org/10.1029/JA083iA06p02637

[2] D. J. Kessler, “Collisional Cascading: The Limits of Population Growth in Low Earth Orbit,” *Advances in Space Research*, 1991. https://doi.org/10.1016/0273-1177(91)90543-S

[3] J.-C. Liou and N. L. Johnson, “Risks in Space from Orbiting Debris,” *Science*, 2006. https://doi.org/10.1126/science.1123785

[4] NASA, *NASA-STD-8719.14C: Process for Limiting Orbital Debris*. https://www.nasa.gov/wp-content/uploads/2018/01/process_for_limiting_orbital_debris.pdf

[5] Federal Communications Commission, *FCC 22-74, Mitigation of Orbital Debris in the New Space Age, Second Report and Order*, 2022. https://docs.fcc.gov/public/attachments/FCC-22-74A1.pdf

[6] ISRO, “India’s Intent on Debris-Free Space Missions — Explained,” 19 April 2024. https://www.isro.gov.in/Debris_Free_Space_Missions.html

[7] S. Wachter, B. Mittelstadt, and C. Russell, “Counterfactual Explanations without Opening the Black Box,” *Harvard Journal of Law & Technology*, 2018. https://jolt.law.harvard.edu/assets/articlePDFs/v31/Counterfactual-Explanations-without-Opening-the-Black-Box-Sandra-Wachter-et-al.pdf

[8] P. Buneman, S. Khanna, and W.-C. Tan, “Why and Where: A Characterization of Data Provenance,” ICDT, 2001. https://doi.org/10.1007/3-540-44503-X_20

[9] S. Parasuraman, T. B. Sheridan, and C. D. Wickens, “A Model for Types and Levels of Human Interaction with Automation,” *IEEE Transactions on Systems, Man, and Cybernetics*, 2000. https://doi.org/10.1109/3477.844354

[10] NIST, *AI Risk Management Framework (AI RMF 1.0)*, 2023. https://www.nist.gov/itl/ai-risk-management-framework